Technical Approach Report

Cynbit DocuFlow AI - Invoice Document Intelligence PoC

Prepared for: Cynbit Technologies - AI/ML Engineer Intern Assessment
Prepared on: 30 October 2026
Implementation: Python, Streamlit, Ollama/Qwen2.5, PyMuPDF, Tesseract, Pydantic, SQLite

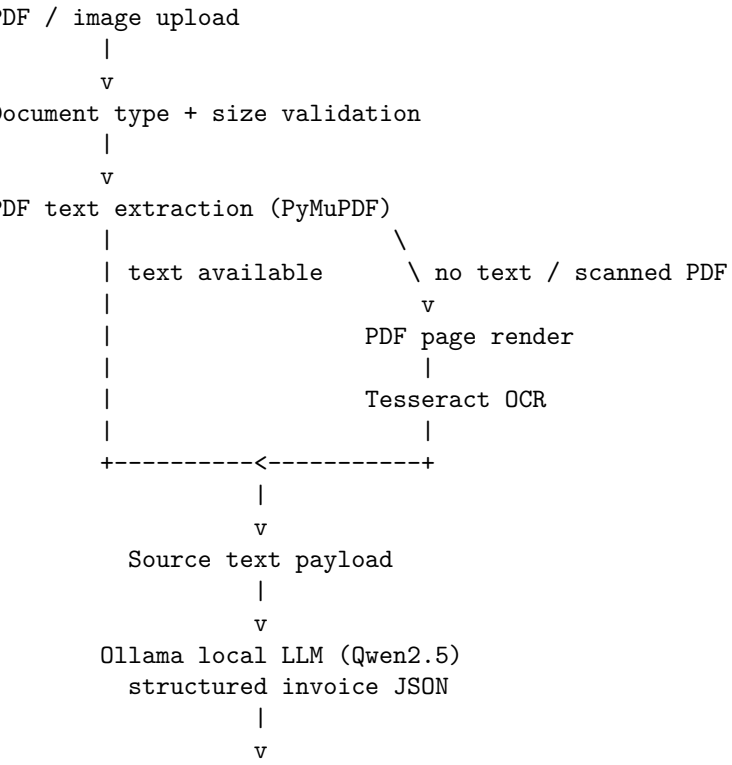
---

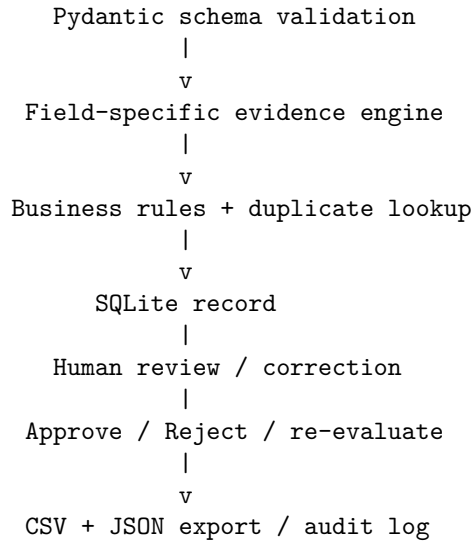
1. Objective

Build a functional demonstration that ingests PDF/image invoices, extracts structured invoice data, verifies values against source text, flags validation exceptions and duplicates, supports human review, preserves approval history, and exports business-ready JSON/CSV.

The system is local-first and uses synthetic invoices. The implementation is a proof of concept, not a production financial-processing service.

2. Architecture overview





3. Technology choices

Layer	Technology	Reason for selection
UI	Streamlit	Rapid functional upload/review/export interface
Language	Python	Common AI/ML and document-processing ecosystem
PDF text and rendering	PyMuPDF	Extract embedded text and render scanned pages
OCR	Tesseract + pytesseract	Local OCR for image documents and scanned PDF pages
LLM inference	Ollama with Qwen2.5 3B	Local structured extraction; avoids depending on a paid cloud API for the demo
Schema validation	Pydantic	Typed invoice structure and JSON validation
Data handling	Pandas	Tabular record display and CSV generation
Persistence	SQLite	Lightweight local record/status/audit storage

Layer	Technology	Reason for selection
Tests	Pytest	Automated regression checks for core rules and workflows

The default model is `qwen2.5:3b`. The application can use another installed Ollama model by setting `OLLAMA_MODEL`.

4. Data pipeline

4.1 Ingestion

The upload layer limits file types to PDF, PNG, JPG/JPEG, and TIFF, with a configured maximum size. Unsupported formats and oversized uploads are rejected before processing.

4.2 Text extraction and OCR

For PDFs, PyMuPDF first extracts embedded text. If no usable text is detected, each page is rendered to an image and Tesseract OCR is applied using the English language pack. Direct image uploads are passed to Tesseract.

The application records the method (`pdf_text`, `pdf_ocr`, or `image_ocr`) so a reviewer can distinguish native text extraction from OCR.

4.3 LLM extraction and prompt design

The prompt instructs the model to return a JSON object, use null for unknown scalar fields, avoid inventing missing values, distinguish the supplier from the `BILL TO` customer, normalize unambiguous dates to ISO format, and return line items as structured objects.

The invoice schema includes:

- Vendor and invoice identifiers.
- Invoice date and due date.
- Currency.
- Subtotal, tax amount, and total amount.
- Payment status.
- Line items with description, quantity, unit price, and amount.
- Extraction notes and evidence metadata.

Pydantic validates the response before downstream logic uses it. Model-generated confidence values are not treated as calibrated probabilities; the deterministic evidence engine overwrites evidence indicators based on source text.

4.4 Field-specific evidence verification

The evidence engine does not accept a value-only substring match. It uses field-specific checks:

- Vendor: match the supplier phrase in a header-like line and reject `BILL TO`, `SHIP TO`, or customer-context lines.
- Invoice number: require an invoice-number label and the expected identifier immediately after it.
- Invoice and due dates: require the corresponding date label and a normalized date value.
- Currency: require an explicit currency context such as `Currency: INR` or `AMOUNT (INR)`.
- Subtotal and total: require the relevant subtotal/grand-total label and amount.
- Tax: match a directly labelled tax amount or aggregate explicitly labelled CGST/SGST/IGST components when the component sum equals the extracted tax amount.
- Payment status: require a payment-status label and supported status token.

The engine returns `Verified`, `Needs review`, or `Not verified` for each field. It is an evidence-presence indicator, not a probability or authenticity verdict.

4.5 Financial and duplicate checks

The validation module checks required invoice fields, checks whether subtotal + tax agrees with total within a small rounding tolerance, and flags invalid date order/format. Duplicate detection checks normalized supplier name + invoice number before inserting a new record.

The current prototype does not calculate every tax treatment or validate line-item arithmetic comprehensively. Human review is required.

5. Human approval, persistence, and audit

SQLite is the workflow-status source of truth. Records carry `Pending Review`, `Approved`, or `Rejected` status. Corrections made in the review UI are saved with the status update in one database transaction.

Evidence re-evaluation is deliberately separate from invoice approval. A reviewer selects an existing record and uploads the source PDF/image. The application re-runs text extraction, checks the stored invoice values against the source, updates only evidence metadata and retained source text, and appends an audit entry. It does not change the invoice business fields or workflow status. The approval gate blocks approval if a required supplier/invoice/date/total field is missing or if any populated scalar field lacks verified source evidence.

Audit entries include the record ID, source filename, workflow status at re-evaluation, prior/new verification metadata, SHA-256 of extracted source text, and timestamp.

Legacy records without stored verification metadata are represented as `Not evaluated`, not falsely counted as verified.

6. Exports

CSV and JSON are generated from the same SQLite snapshot. Both exports include invoice identity, values, workflow status, verification status/summary, and timestamps. JSON retains detailed field evidence and re-evaluation audit details; CSV includes verification counts and audit count/last re-evaluation time.

The export summary has nine scalar fields and counts:

- verified fields
- fields needing review
- fields not verified
- fields not evaluated
- total fields

The counts are derived from the per-field state map, with missing legacy states normalized to `Not evaluated`.

7. Functional test results

The release suite contains 46 automated tests; all 46 passed in the final regression run.

- Document upload validation.
- Invoice schema/confidence constraints.
- Invoice total/date/missing-field validation.
- Duplicate lookup behaviour.
- Direct and label-specific evidence matching.
- OCR-joined context cleanup.
- Tax aggregation and mismatch rejection.
- Approval field/status persistence.
- SQLite source-text migration.
- CSV/JSON workflow consistency.
- Legacy `Not evaluated` counts.
- Evidence re-evaluation status preservation and audit logging.

In a manual Windows test using the bundled synthetic image-only invoice, the application displayed `pdf_ocr` and extracted supplier, invoice number, dates, subtotal, combined tax, total, and payment status. That manual test is a single synthetic example and is not a statistically meaningful accuracy evaluation.

8. Reproducibility and setup

1. Install Python 3.14, create/activate a virtual environment, and install `requirements.txt`.
2. Install Tesseract OCR for Windows and ensure `eng` language data is available.
3. Install/start Ollama and ensure `qwen2.5:3b` is available (`ollama list`).
4. Run `python -m pytest -v`.
5. Start with `streamlit run app.py`.
6. Upload `samples/invoices/DocuFlow_Scanned_Test_Invoice.pdf`; the extraction method should show `pdf_ocr`.

Detailed Windows commands are in the project `README.md`.

9. Limitations and next steps

- Evaluate on a larger, permissioned invoice sample with field-level ground truth.
- Measure extraction precision/recall, OCR failure rate, human correction rate, processing latency, and duplicate alert precision.
- Add robust supplier identity normalization and duplicate override policies.
- Expand source evidence mapping to line items and additional invoice layouts/languages.
- Add role-based access control, encryption/secrets handling, structured logs, secure deployment, and ERP/accounting integrations before production use.
- Consider a layout-aware document model if OCR-only pipelines are insufficient on complex tables.

10. Public alignment references

1. Cynbit AI automation services: https://www.cynbit.in/services (LLM automation for manual data entry/document handling).
2. Cynbit AI/ML Engineer Intern role: https://www.cynbit.in/careers/ai-ml-engineer-intern (document extraction/processing pipelines, prompt engineering, Python/ML/LLM requirements).
3. Cynbit Careers: https://www.cynbit.in/careers.

Public-page content was reviewed on 30 October 2026. No private Cynbit systems, data, or internal process claims are implied.